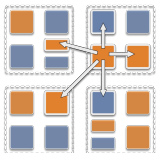


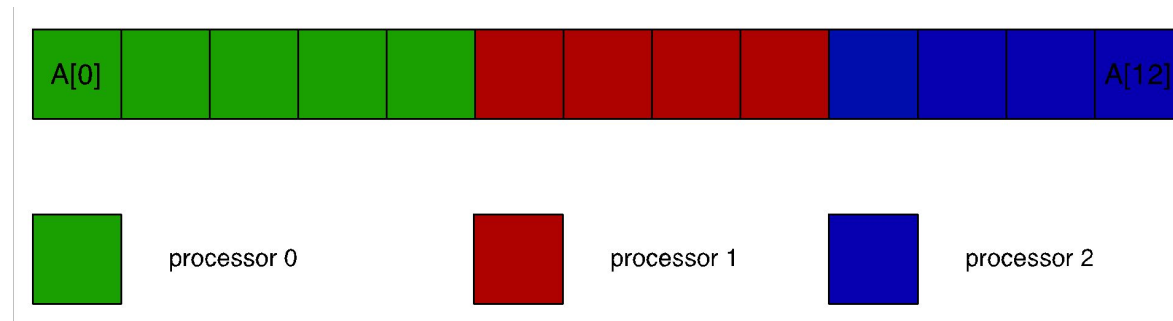
Chare Arrays

- Indexed collections of chares
 - Every item in the collection has a unique index and proxy
 - Can be indexed like an array or by an arbitrary object
 - Can be sparse or dense
 - Elements may be dynamically inserted and deleted
- For many scientific applications, collections of chares are a convenient abstraction
- Instead of creating networks of chares that learn about each other (by sending proxies to each other), each element in a chare array knows about all the others

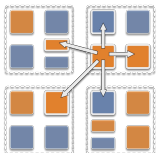


Chare Array Location

- By default, chare arrays are distributed across the available processors



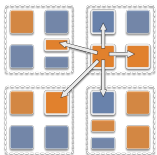
- (This is one possible initial assignment for 1D chare arrays)
- Chare array elements can be migrated by the user or the runtime (load balancing)



Declaring a Chare Array

Chare definition:

```
class Test(Chare):  
    def foo(self):  
        pass  
  
    def sayHi(self):  
        pass
```



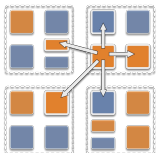
Constructing a Chare Array

- Constructed much like a regular chare
- The size of each dimension is passed to the constructor at the end

```
proxy1d = Array(Test, 4)  
Proxy2d = Array(Test, (2,2))
```

- The proxy represents the entire array, and may be indexed to obtain a proxy to an individual element in the array

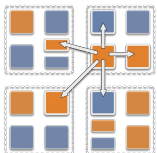
```
proxy1d[1].foo()  
proxy2d[(0, 1)].foo()
```



thisIndex

- N-d: thisIndex returns a tuple of indices of the current chare array element

```
class Test(Chare):  
    def start(self):  
        print('I am element', self.thisIndex, 'on PE', charm.myPe(),  
              'sending a msg to element 1')  
        self.thisProxy[1].sayHi()  
  
    def sayHi(self):  
        print('Hello from element', self.thisIndex, 'on PE', charm.myPe())
```



Chare Array: Hello Example

```
class Hello(Chare):
    def __init__(self, num_elements):
        self.num_elements = num_elements

    def sayHi(self, hello_num):
        my_index = self.thisIndex[ 0]

        print(f"Hi[{hello_num}] from element {my_index} on PE
{charm.myPe()}")

        if my_index == self.num_elements - 1:
            print('All done')
            exit()
        else:
            return self.thisProxy[my_index + 1].sayHi(hello_num + 1)

def main(args):
    num_elements = 5  # default: create a 1D chare array of 5 elements

    print(f"Running Hello on {charm.numPes()} for {num_elements} elements." )

    array_proxy = Array(Hello, num_elements, args=[num_elements])
    array_proxy[ 0].sayHi(17)

charm.start(main)
```

Broadcast

- A message to each object in a collection
- The chare array proxy object is used to perform a broadcast
- It looks like a function call to the proxy object
- From the main chare that created a chare array:

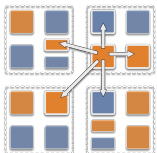
```
proxy1d = Array(Test, 4)  
proxy1d.foo()
```

- From a chare array element that is a member of the same array:

```
self.thisProxy.foo()
```

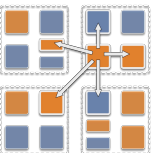
- From any chare that has a proxy p to the chare array

```
p.foo()
```



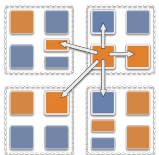
Reduction

- Combines a set of values: sum, max, concat, ...
- Usually reduces the set of values to a single value
- Combination of values requires an operator
- The operator must be commutative and associative
- Each object calls *reduce* in a reduction



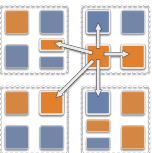
Reduction: Example

```
class Test(Chare):  
    def __init__(self, n):  
        self.num = n  
  
    def work(self):  
        self.reduce(self.thisProxy[0].printResult, self.num, Reducer.sum)  
  
    def printResult(self, result):  
        print('Result =', result)
```



Channels

- Channels in Charm4py are streams or pipes between chares (currently only point-to-point).
- They are useful for writing iterative applications where chares always send/rcv to/from the same the set of chares.



Channels

```
class Demo(Chare):  
    @coro  
    def run(self):  
        peer = self.thisProxy[1 if self.thisIndex[0] == 0 else 0]  
        ch = Channel(self, remote=peer)  
  
        if self.thisIndex[0] == 0:  
            ch.send("Hello from 0!")  
        else:  
            print(ch.recv())  
            charm.exit()
```

- On calling `recv()` a coroutine suspends until there is something to receive.

